

MeshAI

Technical specification. How a phone and a laptop pool their memory to run open AI models neither can run alone.

Version	2.0, 22 September 2026
Status	Design written before the event. Nothing here has been built yet, as the rules require.
Numbers	Every number is either sourced or marked target (to be measured) or estimate (our own arithmetic).
Team	Prajwal, Yuvaraj

Scope in one line. We reuse llama.cpp for inference. We build the part that is missing: a phone app that joins without a terminal, device profiling, a scheduler that decides where each part of a model runs, and one local API for coding tools.

01 Summary

What MeshAI is, and what it is not.

Question	Answer
Problem	New open models are often too big for a budget laptop, and also too big for a phone on its own. Buying a GPU is expensive, and company code often cannot go to cloud AI.
Solution	Pool the free memory of a laptop and one or more phones on a private link, and run the model across them.
Who it is for	Developers with a strong phone and a modest laptop, working on code they cannot or will not send to the cloud.
Main workload	Jobs where a few minutes of waiting is fine: writing tests, reviewing a branch, documenting old code, reading large logs.
What we reuse	llama.cpp (inference, GGUF models, RPC backend, Android and OpenCL support), open models such as gpt-oss-20b and Qwen3-Coder-30B-A3B.
What we build	Phone app with an embedded model worker, QR pairing, device profiler, scheduler, live dashboard, laptop coordinator and local API.
What we do not claim	That splitting a model makes it faster than one device that can already hold it. That the NPU will work on day one. That it works on any Wi-Fi.

02 Problem Statement

12.1 GB

gpt-oss-20b model file (MXFP4). 21B parameters, 3.6B used per token.

~4 GB

Free RAM on a typical 8 GB laptop once the OS and a browser are running (estimate).

~10 GB

Usable RAM on a 16 GB phone. PowerInfer-2 measured 11 GB free on a 16 GB phone.

Neither device can load the model alone. Together they can. With a second phone, the 18.6 GB Qwen3-Coder-30B-A3B (Q4_K_M) also fits.

Memory	Peak bandwidth
Laptop, DDR4-3200, one channel	25.6 GB/s
Laptop, DDR4-3200, two channels	51.2 GB/s
iQOO 15, LPDDR5X at 5300 MHz	about 85 GB/s

Generating each token means reading the model's active weights from memory, so speed is roughly bandwidth divided by bytes read per token (Pope et al. 2022; MELTing Point 2024). On these numbers the phone is the faster device.

Sources: JEDEC DDR4 figures; Qualcomm Snapdragon 8 Elite Gen 5 product brief; Hugging Face GGUF files for both models; PowerInfer-2, arXiv 2406.06282.

03 What Makes It Different

Existing work

The hard parts of inference exist upstream. What is missing is a product that makes phones and laptops work together without manual setup.

Project	What it does today	What it means for us
llama.cpp RPC	Lets one machine use other machines' memory and compute. Splits weights by free memory. Can cache tensors locally with -c. The README calls it a proof of concept, fragile and insecure on open networks.	Our inference layer. We only run it on a private link we control.
llama.cpp on Android	Builds with the Android NDK. <code>ggml_backend_rpc_start_server()</code> is a library call, so a worker can run inside an app.	The phone needs no terminal or Termux. The worker lives inside our app.
llama.cpp OpenCL	Qualcomm Adreno backend. Adreno 840 (Snapdragon 8 Elite Gen 5) is listed as supported. Q4_0 is recommended.	GPU path to test. Not assumed to be faster: on an 8 Elite tablet, CPU decode beat GPU for a 7B model (discussion #23736).
llama.cpp Hexagon (NPU)	Experimental. About 3.5 GB per NPU session. Issue #25876 reports garbled output on SM8850, the chip in the iQOO 15.	We test it and fall back automatically. Not a dependency.
prima.cpp	Research system for home device clusters over Wi-Fi, including Android phones.	Evidence that pooling works for models too big for one device. Its scheduler also skips weak devices.
Exo	Automatic discovery and partitioning, focused on Apple and desktop clusters.	Shows the value of zero setup. Different target hardware.
AI Doomsday Toolbox, Synapse, Meshlit	Community projects for phone clusters, browser based splitting and phone task sharing.	Proof that people want this. We have not verified their numbers, so we do not quote them.

Published results

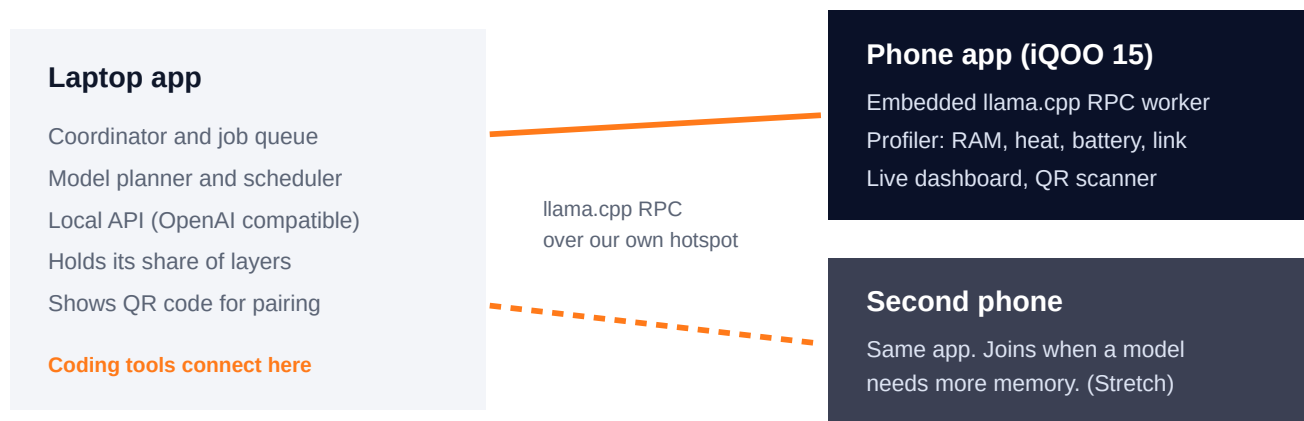
What peer reviewed and preprint research says about splitting models across edge devices.

System	Setup	Result
prima.cpp arXiv 2504.08791	4 home devices on Wi-Fi (3 to 7 ms latency), including phones	8B and 14B: no gain over one device. 30B: 72 vs 202 ms/token. 70B: 674 vs 10,120 ms/token.
TPI-LLM arXiv 2410.00531	4 laptops on home Wi-Fi	70B at 26.1 s/token with 3.1 GB per device. Plain pipeline splitting leaves devices idle.
EdgeShard arXiv 2405.14371	Jetson Orin boards and one RTX 3090, 1 Gbps	Llama2 7B: 75.9 vs 140.3 ms/token. Up to 2x throughput.
LinguaLinked arXiv 2312.00388	4 Android phones, models up to 3B	1.1 to 1.6x over their own baseline split, about 3.6 s/token.
MELTing Point MobiCom 2024	Phones running llama.cpp and MLC	Decode is memory bound. Throughput drops in steps as phones heat up.
llama.cpp #22850	RPC over wired 2.5 GbE	Decode fell from 180 to between 80 and 130 tokens/s. The loss came from protocol and sync overhead, not bandwidth.

What we take from this. Pooling helps when a model does not fit on one device. It does not help when it does. So MeshAI runs a model on a single device whenever it fits, and only splits when it must. We found no published phone plus laptop measurements for this setup. We plan to publish ours.

04 Architecture

Two apps. Everything runs on devices the user owns.



Component	Job	Hackathon version
Phone app	Joins the mesh, reports its state, runs its share of the model, shows the dashboard.	Kotlin app, llama.cpp through the NDK, foreground service.
Profiler	Free memory, backend speed (CPU, GPU, NPU), temperature, battery, link latency and throughput.	Android APIs plus a short benchmark on join.
Model planner	Reads the GGUF file, works out bytes per layer, context memory and headroom, and proposes a placement.	Python or Go on the laptop.
Scheduler	Chooses the plan (section 06) and drops devices that would hurt.	Rules based on measured numbers.
Local API	One stable address for tools, whatever the device layout is.	llama.cpp server with RPC backends behind /v1/chat/completions.
Job queue	Runs long jobs (tests, reviews, docs) and keeps results.	Simple queue with progress shown on the phone.

Phone app

- **Worker inside the app.** llama.cpp built with the NDK, with the RPC server started through `ggml_backend_rpc_start_server()`. No terminal, no Termux, no IP addresses.
- **Foreground service with the screen on.** Android's low memory killer targets background apps first, and Wi-Fi low latency mode only works in the foreground. The dashboard is the foreground screen.
- **QR pairing with the camera.** The laptop shows a code with its address and a one time key. The phone scans it and joins.
- **Model cache.** The phone's share of the model is stored locally and checked by hash, so it is not resent every session. Office Kit file transfer can preload it.
- **Backends.** CPU first. Adreno OpenCL tested next. Hexagon NPU tested as an experiment with automatic fallback.
- **Dashboard.** Layers held, tokens per second, temperature, battery, link quality and job progress.

Laptop app

- **Pairing.** Shows the QR code and only accepts devices that scanned it.
- **Model planner.** Reads GGUF metadata, estimates weight and context memory per layer, and proposes a placement.
- **Execution.** Starts llama.cpp with the chosen RPC workers and exposes one local API.
- **Fallback.** If a phone disappears, stop cleanly, replan with the devices left, and restart the job from where it can. Live migration in the middle of a token is not a goal.

05 Tech Stack

Where our docs left a choice open, both options are listed. We will pick one before the event.

Layer	Option A	Option B
Phone app	Kotlin with Jetpack Compose	Flutter with a native plugin
On device inference	llama.cpp built with the Android NDK (C++)	llama.cpp in Termux, for quick tests only
Phone compute	CPU (ARM)	Adreno GPU via OpenCL, Hexagon NPU (experimental)
Model splitting	llama.cpp RPC backend	None needed when the model fits one device
Laptop coordinator	Python (FastAPI)	Go or Node.js
Local API	llama.cpp server, OpenAI compatible	Thin proxy in the coordinator
Models (GGUF)	gpt-oss-20b (MXFP4)	Qwen3-Coder-30B-A3B (Q4_K_M), small models for tests
Pairing and link	QR code, our own hotspot	USB tethering, Wi-Fi Direct later
Phone and laptop bridge	vivo Office Kit	None

06 Scheduling

Execution plans

One scheduler, several plans, chosen by what the user needs.

Situation	Plan	How it works	Status
Model fits on one device	Single device	Run it on the fastest device that can hold it, usually the phone. No network in the path.	Build
Model too big for one device	Layer split	Contiguous blocks of layers on each device. Weights and their context memory stay put. Only the hidden state moves between devices.	Build
Many independent jobs	Parallel jobs	Each device runs its own job with a model that fits it, for example reviewing different files.	Stretch
Large mixture of experts model	Expert placement	Place experts on different devices and route tokens to them.	Later
Split inside each layer	Tensor parallel	Needs a sync inside every layer. Too chatty for Wi-Fi.	Not over Wi-Fi

Scheduler rules

Signal	Source on the phone	Rule
Free memory	ActivityManager memory info, llama.cpp device memory query	Hard limit. Keep headroom so Android does not kill the app.
Backend speed	Short benchmark on join (CPU, OpenCL, Hexagon)	Use the fastest backend that gives correct output.
Link latency and throughput	Ping and a short transfer test	Latency decides generation cost, throughput decides prompt cost.
Temperature	PowerManager thermal status and thermal headroom	Reduce the phone's share or pause before it throttles hard.
Battery and charging	BatteryManager	Prefer a charging phone. Respect a user set minimum.
Reliability	Worker health checks	Drop a device that keeps failing.

Core rule. Use the fewest devices that can hold the model. Add a device only if the memory it brings is needed, and leave it out if it would slow everything down.

07 Technical Feasibility

Why newer models suit a memory pool

Mixture of experts (MoE) models store many experts but use only a few for each token. They need a lot of memory to hold, but each token reads only a small part of it. That is exactly the case where pooling memory helps and the cost of splitting stays small.

Model	File	Total parameters	Used per token	Rough bytes read per token (estimate)
gpt-oss-20b (MXFP4)	12.1 GB	21B	3.6B	about 2 GB
Qwen3-Coder-30B-A3B (Q4_K_M)	18.6 GB	30.5B	3.3B	about 2 GB
A dense 20B model (Q4)	about 11 GB	20B	20B	about 11 GB

Speed estimate for gpt-oss-20b on a laptop plus one phone. About a third of the weights sit on the laptop and two thirds on the phone. Reading about 0.7 GB at roughly 20 GB/s on the laptop and 1.3 GB at roughly 50 GB/s on the phone gives about 60 ms per token, a ceiling of about 15 tokens/s. RPC overhead cost 28 to 55% even on wired LAN, and Wi-Fi adds a round trip per token. **Our target is 3 tokens/s or more, measured on the first day.** This is enough for background jobs, which is why we target them.

What crosses the network

Using Qwen3-Coder-30B-A3B as the example (48 layers, hidden size 2048, 4 KV heads, head size 128):

Quantity	Calculation	Meaning
Hidden state per token, per boundary	$2048 \text{ values} \times 2 \text{ bytes} = 4 \text{ KB}$	Tiny. Latency matters more than bandwidth when generating.
Prompt of 4K tokens, per boundary	$4096 \times 2048 \times 2 \text{ bytes} = 16 \text{ MB}$	Reading a long prompt needs bandwidth.
Context memory (KV cache) per token	$48 \times 2 \times 4 \times 128 \times 2 \text{ bytes} = 96 \text{ KB}$	Stays on the device that owns the layers.
KV cache at 16K tokens	about 1.5 GB	Long context eats pooled memory. The planner must count it.

Estimates at FP16. Real traffic is larger because of protocol overhead. Model shape from the Qwen3-Coder-30B-A3B model card and config.

Network

Link	Role	Notes
Our own hotspot or pocket router	Default for the event	Guest and venue Wi-Fi often isolates clients, which blocks device to device traffic (Cisco Meraki docs).
USB tethering	Backup	Stable link if Wi-Fi is crowded. We will measure its latency.
Home Wi-Fi	Normal use	Research setups saw 3 to 7 ms latency (prima.cpp, TPI-LLM).
Busy office Wi-Fi	Avoid for splitting	Latency has a long tail: about 20 ms at p90 and 250 ms at p99 (Sui et al., MobiSys 2016).
Internet	Never for splitting	Only for independent jobs, and only in a future version.

08 Office Kit

Office Kit is vivo's phone and laptop bridge (screen mirroring, shared clipboard, file transfer, remote control). It is not a compute channel, so model traffic uses our own link. We use Office Kit where it genuinely fits:

- **File transfer** to move multi GB model files to the phone.
- **Screen mirroring** to show the phone dashboard on the laptop during demos.
- **Remote control** to control the phone from the laptop keyboard.

09 Security

Risk	Hackathon version	Later
Anyone on the network can reach the RPC port	Private hotspot only. Pairing by QR key. Port never exposed to the internet.	Mutual authentication and an encrypted link.
Tampered model files	Known hashes for every model we ship.	Signed manifests.
Private code leaving the devices	Nothing is sent outside the paired devices.	Clear permissions and an audit log.

10 Risks and Limits

Risk	What we do about it
The split is too slow	Target background jobs. Use single device or parallel jobs when they win.
The phone GPU or NPU path fails	CPU first. GPU and NPU only if they measure faster and give correct output.
Android kills the worker	Foreground service, memory headroom, smaller context.
The phone overheats	Thermal aware scheduling, charging, a cap on the phone's share.
Venue Wi-Fi blocks devices	Our own hotspot, USB tethering as backup.
Loaner phones are the 12 GB model	Use the second phone earlier, or a smaller quantisation.
Slow model loading	Local cache, preloaded files, llama.cpp - c tensor cache.

What we claim

We claim

- Models too big for each device can run across both.
- Only a few KB cross the network per token per boundary.
- The phone is often the stronger AI device.
- A scheduler that knows when not to split.

We don't claim

- Faster than one device that can hold the model.
- Cloud speed.
- NPU acceleration before we measure it.
- Works on any network.

11 Implementation Plan

Scope

Tier	Items
Build	Laptop plus one iQOO 15 running gpt-oss-20b as a split model. Phone app with embedded worker, QR pairing, profiler and dashboard. Scheduler choosing single device or split. Local API used by a coding tool. One background job type end to end.
Stretch	Second phone and Qwen3-Coder-30B-A3B. Parallel jobs. OpenCL backend if it measures faster.
Later	Hexagon NPU backend, expert placement for MoE models, Wi-Fi Direct, encrypted links.

Measurement plan

Step	Setup	Model	We record
A	Laptop alone	gpt-oss-20b	Does it load? If it swaps, tokens/s.
B	Phone alone	Small model, then gpt-oss-20b	Free RAM, CPU vs GPU tokens/s.
C	Laptop plus one phone	gpt-oss-20b	Time to first token, tokens/s, link use.
D	Laptop plus two phones	Qwen3-Coder-30B-A3B	Same, plus the cost of the extra hop.
E	Best setup, 10 minutes	Same	Temperature, throttling, battery drain.
F	Parallel jobs	Smaller models	Jobs per hour against one device.

Every result is reported with model, quantisation, context length, backend and link type, so anyone can repeat it.

12 References

1. llama.cpp RPC backend. github.com/ggml-org/llama.cpp/blob/master/tools/rpc/README.md
2. llama.cpp ggml-rpc.h (ggml_backend_rpc_start_server). github.com/ggml-org/llama.cpp/blob/master/ggml/include/ggml-rpc.h
3. llama.cpp OpenCL backend. github.com/ggml-org/llama.cpp/blob/master/docs/backend/OPENCL.md
4. llama.cpp Snapdragon Hexagon backend. github.com/ggml-org/llama.cpp/blob/master/docs/backend/snapdragon/README.md
5. llama.cpp issues and discussions #22850, #25876, #11957, #9136, #23736.
6. Li et al. prima.cpp. arXiv 2504.08791.
7. Li et al. TPI-LLM. arXiv 2410.00531.
8. Zhang et al. EdgeShard. arXiv 2405.14371.
9. Zhao et al. LinguaLinked. arXiv 2312.00388.
10. Laskaridis et al. MELTING Point. MobiCom 2024. arXiv 2403.12844.
11. Xue et al. PowerInfer-2. arXiv 2406.06282.
12. Pope et al. Efficiently Scaling Transformer Inference. arXiv 2211.05102.
13. Sui et al. Characterizing and Improving WiFi Latency in Large-Scale Operational Networks. MobiSys 2016.
14. OpenAI gpt-oss-20b model card and ggml-org/gpt-oss-20b-GGUF. huggingface.co
15. Qwen3-Coder-30B-A3B-Instruct model card and unsloth GGUF files. huggingface.co
16. Qualcomm Snapdragon 8 Elite Gen 5 product brief. qualcomm.com
17. iQOO 15 specifications. iqoo.com/in/products/param/iqoo15
18. iQOO Hackathon 2026 guide. iqoo.reskilll.com/guide
19. Cisco Meraki, Wireless Client Isolation. documentation.meraki.com
20. Android developer docs: memory management, low memory killer, PowerManager thermal API, WifiManager low latency mode.